Production Checkpoint Report

FireSVM · MobileNet-SSD · Thermo-X3D T5v2

Smart Thermal System for Patient Safety Monitoring

Guy Chen   Yaniv Blau
Roy Lieberman
Supervisor: Or Zilberberg   Advisor: Dr. Oshrit Hoffer   Afeka College of Engineering

July 4, 2026

Abstract

This report documents the three model checkpoints deployed in the live demonstration system: the **FireSVM** fire detector, the **MobileNet-SSD** person detector, and the **Thermo-X3D T5v2** contact (touch) detector. For each model we describe the architecture, inputs and outputs, feature extraction, loss and optimizer, the heuristics that surround the network, benchmark accuracy (accuracy/precision/recall/F_1/FAR), and measured CPU runtime with estimates for the Raspberry Pi 5 target. All sensing is thermal-only (Waveshare 80×62 module, 8 Hz) — no optical imagery is ever captured.

1 System context

Three thermal cameras observe the room. Each 8 Hz tick produces three raw 62×80 frames in °C. Two preprocessing views feed the models (an empirically-validated split — see §4.4):

- **Raw frames** go to the per-camera detectors (FireSVM, MobileNet-SSD). Raw input transfers across rooms without recalibration.
- **Tateno residuals** (Gaussian smooth → background subtract → rectified L_1 residual) against a *session-local rolling 25th-percentile background* go to Thermo-X3D. The p25 background is computed from the last ∼30 s of frames and refreshed continuously — no per-room calibration, ∼2 s warm-up.

Task	Checkpoint file	Format	Size
Fire	`fire_svm_detector/_default.thalg`	sklearn pickle	64 KB
Person	`mobilenet_ssd_detector/Waveshare_26984_raw.onnx`	ONNX	0.27 MB
Contact	`thermo_x3d_detector/Waveshare_26984_T5_v2.ftz.onnx` (+meta)	ONNX	1.6 MB

Table 1: Deployed checkpoints. All are committed on the `demo-linux` branch; total ≈2 MB. The two neural networks run via `onnxruntime` — the deployment is torch-free.

2 FireSVM — fire detection

2.1 What is an SVM?

A support vector machine is a binary classifier that finds the separating hyperplane $w^\top \phi(x) + b = 0$ with the *maximum margin* — the largest distance to the nearest training points of either class. Only those nearest points (the *support vectors*) determine the boundary, which makes the model compact and robust. Two extensions make SVMs practical:

- **Soft margin (C).** Real classes overlap, so slack variables allow some points inside the margin; C trades margin width against training violations (C=1 here).

- **Kernel trick.** Instead of computing $\phi(x)$ explicitly, a kernel $k(x, x') = \langle \phi(x), \phi(x') \rangle$ evaluates inner products in a high-dimensional space. We use the radial basis function kernel $k(x, x') = \exp(-\gamma \|x - x'\|^2)$ with $\gamma = \text{scale} = 1/(d \cdot \text{Var}(X))$, which yields a smooth non-linear decision boundary.

Prediction is $\hat{y} = \text{sign}(\sum_i \alpha_i y_i \, k(x_i, x) + b)$ over the support vectors x_i. A logistic fit on the decision values (*Platt scaling*) turns the raw margin into the confidence probability we report.

2.2 What is special about our FireSVM

FireSVM is not applied to pixels. A classical segmentation stage proposes a region, and the SVM classifies six scalar statistics of that region — which makes the model *resolution-invariant* (the same checkpoint works on 32×24 and 80×62 sensors) and extremely fast.

Pipeline (per raw frame).

1. **Region proposal — Otsu segmentation.** The frame is histogrammed (256 bins); Otsu's criterion selects the threshold maximizing between-class variance; the binary mask is shaped by one 3×3 erosion and two dilations.
2. **Feature extraction.** Connected components are extracted and the *hottest blob* is selected. Six features describe it: max temperature, mean, standard deviation, area, skewness and kurtosis of the blob's pixel distribution. (Fire blobs are small, extremely hot and heavy-tailed; people are large, near 36°C and symmetric.)
3. **Standardization.** Features are z-scored with the μ, σ learned at fit time.
4. **Classification.** RBF-SVC (C=1, γ=scale, 874 support vectors) outputs SAFE or ACTIVE_COMBUSTION with a Platt confidence.

Input/output. In: one raw 62×80 float32 frame (°C). Out: a `FireAlert` — level, Platt confidence, and the hottest blob's features including its bounding box (drawn in the demo UI).

Training. Frames with a YOLO class-0 (fire) box are positives; all other labeled frames are SAFE. Stratified sequential 70/30 split per (scene, camera, class). No gradient training — the SVC solves its convex dual (SMO); there is no optimizer/epoch schedule, and regularization is the C soft margin.

Runtime note. Profiling showed 88% of inference time was *scipy call overhead* for skewness/kurtosis ($\sim$13 blobs × 2 calls per frame). Replacing them with an algebraically identical numpy implementation (verified bit-identical on 1,395 real blobs) cut predict from 8.8 to **1.19 ms/frame** with zero behavioral change.

	Accuracy	Precision	Recall	F_1	FAR	TP	TN	FP	FN
FireSVM (test)	96.4%	99.7%	75.5%	85.9%	0.04%	314	2410	1	102

Table 2: FireSVM benchmark — 2,827 held-out test frames (416 fire-positive) from the 70/30 protocol; 6,497 training frames. FAR is the false-alarm rate over fire-free frames. The near-zero FAR is the design priority for a ward: the detector essentially never cries wolf, at the cost of missing low-contrast ignition frames (small cigarette embers at distance).

3 MobileNet-SSD — person detection

3.1 Architecture

A single-stage anchor-based detector (SSD) on a MobileNet-style backbone, sized for our tiny native input — we do *not* upscale the thermal frame to 300×300 like standard SSD; the network runs on $1 \times 62 \times 80$ directly. The backbone is built from **depthwise-separable convolutions** (depthwise

3×3 + pointwise 1×1, each with BatchNorm+ReLU), the MobileNet trick that cuts multiply–accumulates roughly 8–9× versus dense convolutions.

Stage	Operator	Stride	Output ($C \times H \times W$)
Input			$1 \times 62 \times 80$
Stem	Conv 3×3 + BN + ReLU	2	$16 \times 31 \times 40$
Block 1	DW-separable	1	$32 \times 31 \times 40$
Block 2	DW-separable	2	$64 \times 15 \times 20$ (→ **F1**)
Block 3	DW-separable	1	$64 \times 15 \times 20$
Block 4	DW-separable	2	$128 \times 7 \times 10$ (→ **F2**)
Block 5	DW-separable	1	$128 \times 7 \times 10$
Head (F1)	Conv 3×3 → K(4+2) ch	1	box $12 \times 15 \times 20$, cls $6 \times 15 \times 20$
Head (F2)	Conv 3×3 → K(4+2) ch	1	box $12 \times 7 \times 10$, cls $6 \times 7 \times 10$

Table 3: MicroMobileNet-SSD, Waveshare configuration. Two detection scales: F1 (fine, near people) and F2 (coarse, large/close people). K=3 anchors per cell ⇒ $15{\cdot}20{\cdot}3 + 7{\cdot}10{\cdot}3 = 1110$ anchors total.

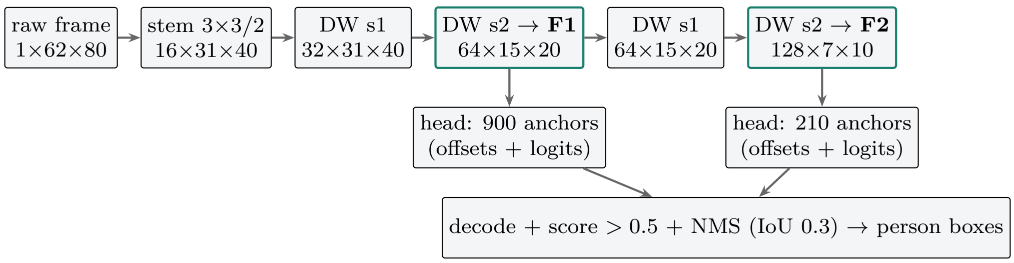

Figure 1: Data flow. (Block 5, a stride-1 refinement after F2, omitted for clarity.)

Anchors. Each feature-map cell holds K=3 anchor boxes with aspect ratios {1:1, 1:2, 1:3} (square, tall, very tall — person-shaped) and base widths of 10 px (F1) and 16 px (F2) in input coordinates.

Input/output. In: one raw frame, per-frame z-scored $(x-\mu)/(\sigma+10^{-6})$. Out: for each of the 1110 anchors, 4 box offsets and 2 class logits. Offsets are decoded against the anchor (cx, cy scaled by variance 0.1; w, h by $e^{0.2\,d}$), softmax scores are thresholded at 0.5, and greedy non-maximum suppression (IoU 0.3) yields the final `(x,y,w,h,score)` person boxes.

3.2 Loss, optimizer, heuristics

Loss is the standard SSD multi-task objective. Anchors are matched to ground truth by IoU (positive ≥ 0.5, negative < 0.4, in-between ignored):

$$\mathcal{L} = \frac{1}{N_{pos}}\Big(\underbrace{\sum \mathrm{CE(cls)}}_{\text{pos + hard negatives}} + \lambda \underbrace{\sum \mathrm{SmoothL1(loc)}}_{\text{positives only}}\Big), \qquad \lambda = 1.$$

Hard-negative mining keeps only the 3 highest-loss background anchors per positive (3:1), preventing the 1110-to-few anchor imbalance from drowning the signal. **Optimizer:** Adam, lr 10^{-3}, weight decay 10^{-4} (the only explicit regularizer besides BatchNorm), 50 epochs. **Augmentation:** random horizontal flips and small random translations (zero-padded), boxes transformed accordingly.

3.3 Results

Benchmarked on the Waveshare dataset with an 80/10/10 timeline split (frame-level presence; box quality as mean IoU of matched detections):

Detector (test)	Accuracy	Precision	Recall	F_1	FAR	mean IoU
AdaptiveThreshold	76.6%	93.4%	80.7%	86.6%	83.0%	0.565
HOG+SVM	79.8%	92.7%	85.1%	88.7%	98.1%	0.585
MobileNet-SSD	**95.8%**	**99.1%**	**96.4%**	**97.7%**	**13.2%**	**0.673**

Table 4: Person detection, 825 test frames (772 with people). FAR is computed over the 53 person-free test frames (7 false alarms). The deployed checkpoint is the *raw-input* variant of the same architecture and training recipe: a preprocessing ablation showed the SSD performs best on raw frames (and transfers across rooms), while background-subtracted residuals belong to the contact model.

For deployment the network was exported to ONNX with the anchor decode and NMS re-implemented in numpy; parity vs. the PyTorch path was verified at 7.6×10^{-6} max box/score difference over 330 boxes.

4 Thermo-X3D T5v2 — contact (touch) detection

4.1 Architecture

Contact between people is a *spatio-temporal* event — two warm blobs approaching, merging, and interacting over seconds. Thermo-X3D is a small video CNN inspired by the X3D family: it processes a sliding window of T=5 consecutive residual frames from all three cameras and outputs one contact probability. It is *pixel-based end-to-end*: no bounding boxes, no cross-camera geometry, and crucially **no homography or per-room calibration** (a hard product constraint — 800 rooms).

Every convolution is **(2+1)D-factorized**: a spatial 1×3×3 kernel followed by a temporal 3×1×1 kernel. Factorization halves parameters and FLOPs versus full 3^3 kernels, doubles the non-linearities, and — practically — means every op is effectively a 2-D convolution, which CPU inference libraries optimize well.

Stage	Operator	Spatial stride	Output ($C \times T \times H \times W$)
Input (per camera)			$1 \times 5 \times 62 \times 80$
Stem	Conv 1×3×3 + BN + ReLU	1	$24 \times 5 \times 62 \times 80$
ResBlock 1	(2+1)D + attention + skip	1	$24 \times 5 \times 62 \times 80$
ResBlock 2	(2+1)D + attention + skip	2	$48 \times 5 \times 31 \times 40$
ResBlock 3	(2+1)D + attention + skip	2	$96 \times 5 \times 16 \times 20$
Pool + proj	global avg pool, Linear 96→128		128
Fusion	concat 3 camera streams		384
Head	Linear 384→64, ReLU, Linear 64→2		2 logits

Table 5: Thermo-X3D T5v2. Three *independent* per-camera encoder streams (late fusion). 382,562 parameters total.

Each residual block is: spatial conv → BN/ReLU → temporal conv → BN/ReLU → **thermal attention** → add skip (1×1×1 conv when shape changes) → ReLU. The attention module is CBAM-style, adapted to thermal statistics where *maxima* matter (hot spots):

- *Channel attention:* global average- **and** max-pool over (T, H, W), each through a shared 2-layer MLP (reduction 4), summed, sigmoid-gated per channel.
- *Spatial attention:* channel-wise mean and max maps concatenated, convolved 1×7×7, sigmoid-gated per location.

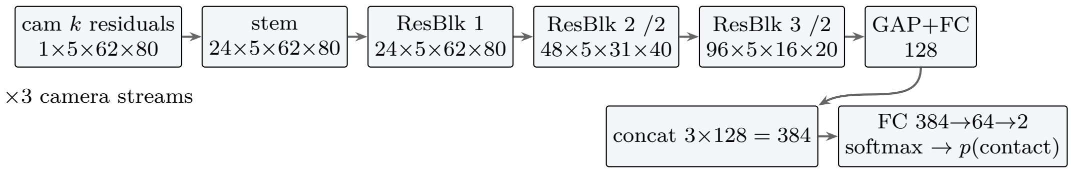

Figure 2: One of three per-camera streams; features fuse late.

Inputs/outputs. In: for each camera, the Tateno residual against the rolling p25 background, globally z-scored (μ=0.868, σ=0.786, statistics of the training pixels); a rolling buffer holds the last T=5 frames per camera, so each 8 Hz tick re-evaluates the network on a $(3, 5, 62, 80)$ volume spanning 0.625 s. Out: softmax contact probability. **Decision heuristics:** alarm threshold 0.35 (selected by validation sweep), evaluated every tick; the pipeline supports N-frame persistence (deployed as benchmarked: N=1).

4.2 Loss, optimizer, training data

Loss: cross-entropy with *inverse-frequency class weights* (positives are ∼35% of frames; weights $n/(2n_{neg}), n/(2n_{pos})$). **Optimizer:** Adam, lr 10^{-3}, weight decay 10^{-4}, 10 epochs, batch 8, over all T-frame windows of contiguous training runs. **Data:** 57 scenes / 30,279 frames / 10,516 positive — the home recordings (re-annotated after a label audit) plus the 06-30 ward-archive segments; per-scene timeline split 60/15/25; the human-labeled `2men_clash` scene is excluded entirely as a held-out benchmark. Training uses the same p25-background residuals as deployment (v2's key fix — the model trains on exactly what it sees at runtime).

4.3 Results

	Accuracy	Precision	Recall	F_1	FAR
In-domain test (7,592 frames, th 0.35)	91.3%	85.6%	90.1%	87.8%	8.0%
Held-out `2men_clash`, frame-exact	—	23.1%	62.2%	33.7%	54.4%
Held-out `2men_clash`, ±2 frames	—	28.1%	73.7%	40.7%	51.3%

Table 6: Thermo-X3D T5v2. The held-out gap is a *label-granularity* artifact, not pure model error: much of the training corpus carries episode-level interval labels (contact marked continuously over multi-minute episodes), teaching episode semantics that frame-exact scoring penalizes. At the *episode* level the model detects 5/5 contact episodes in the held-out scene with 12.8 s of total false-positive time. Frame-exact tightening is the main open item for the next retrain (interval-core training or threshold recalibration).

ONNX backend verification. The deployed ONNX checkpoint was re-scored on the full protocol: fp32 ONNX reproduces PyTorch *exactly* (F_1 87.8% / 33.7%). An int8-quantized variant was also produced (test F_1 86.6%, FAR 8.0→5.2%) but is *slower* than fp32 on x86 (no optimized int8 3-D conv path in onnxruntime) and is kept only as a Pi-side candidate for future testing.

4.4 The classical alternative — "config-D" rule

Alongside the network we maintained a hand-crafted contact rule, the best of a long ablation series (**config-D**): run MobileNet-SSD on *raw* frames; segment warm blobs on the Tateno *residual*; fire when *exactly two* person boxes fall inside one connected warm component (the two-body merge test — rejects person+object and 3-person crowds); then clean the per-frame decision stream with temporal morphological opening (2) and closing (7). This ablation also produced the raw/residual preprocessing split now used system-wide.

On clean labels config-D and Thermo-X3D are nearly tied in-domain (F_1±2: 76.9% vs. 78.5%), but on the held-out room the network won decisively (73.1% vs. 45.3% for the earlier T5 checkpoint)

— the rule's blob-area constants are room-tuned, which conflicts with the no-calibration constraint. Config-D therefore remains a validation baseline; the deployed detector is the network.

5 Runtime

Measured on the development PC (Intel i7-11700K class, 8 cores; single inference, real frames). Pi 5 estimates scale the thread-matched measurements by the per-core gap (Cortex-A76 2.4 GHz vs. ∼4.9 GHz; factor ≈3–3.5×), pending on-device validation.

Model (per frame / per window)	PC CPU	PC (4 threads)	Pi 5 estimate
Tateno preprocessing	0.02 ms	—	∼0.1 ms
FireSVM	1.19 ms	—	3.5–5 ms
MobileNet-SSD	2.6 ms	1.8 ms	8–10 ms
Thermo-X3D (torch fp32, before work)	93.7 ms	—	—
Thermo-X3D (ONNX fp32)	34.5 ms	—	—
Thermo-X3D (ONNX fp32, denormals flushed)	**12.9 ms**	14.7 ms	**45–60 ms (4 cores)**
Thermo-X3D (ONNX int8)	60.8 ms	—	untested
Full tick: 3×(pre+fire+person) + contact	≈24 ms		≈65–90 ms
8 Hz budget	125 ms		125 ms

Table 7: CPU inference runtime. The X3D speedup chain is 7.3× total: ONNX export (2.7×, bit-identical F_1), then flushing *denormal weights* — 19.3% of the trained weights were subnormal floats ($\sim 10^{-41}$, a weight-decay artifact) that trigger CPU microcode penalties; zeroing them in the model changed no output bit (max $|\Delta\text{logit}| = 0$) and gave another 2.7×. Future retrains should clamp subnormal weights post-training.

In the demo application the three per-camera stacks run on parallel worker threads and X3D on a fourth, with depth-1 queues that drop stale frames; the estimated Pi tick (∼65–90 ms) fits the 125 ms budget at 8 Hz with margin. All numbers await confirmation on the Pi itself; the benchmark scripts run there unmodified.

6 Reproducibility

- Benchmarks: `scripts/eval_waveshare_fire.py`, `eval_waveshare_human.py`, `eval_x3d_backends.py`, `benchmark_inference_cpu.py`, `profile_fire_svm.py`.
- Result files (`reports/`): `waveshare_fire_results.json`, `waveshare_human_results.json`, `x3d_T5_v2_results.json`, `x3d_backend_comparison.json`, `cpu_inference_benchmark.json`, `x3d_onnx_runtime.json`.
- Deployment: branch `demo-linux` (weights included); live demo `python -m demo.app -live`.